

Programowanie obiektowe

Wykład 10

Tworzenie aplikacji z graficznym interfejsem użytkownika w Java Swing

Swing

- Swing to biblioteka graficzna w Javie do tworzenia interfejsów użytkownika (GUI).
- Jest częścią standardowego zestawu klas JDK (pakiet `javax.swing`).
- Umożliwia tworzenie okien, przycisków, pól tekstowych, tabel, list, itd.
- Bazuje na starszej bibliotece AWT, ale jest bardziej rozbudowany, nowocześniejszy i w pełni w Javie.
- Komponenty Swing są „lightweight” – niezależne od systemu operacyjnego.

Główne cechy:

- Swing opiera się na komponentach, takich jak:
 - `JFrame` – główne okno aplikacji,
 - `JPanel` – kontener do grupowania innych komponentów,
 - `JButton` , `JLabel` , `JTextField` , `JList` , `JTable` – podstawowe elementy interfejsu.
- Obsługa zdarzeń (np. kliknięcie, wpisanie tekstu)
- Layouty do rozmieszczania komponentów (`FlowLayout` , `GridLayout` , `BorderLayout` , `GridBagLayout`)
- Jest elastyczny i rozszerzalny – można pisać własne renderery, Look & Feel, itp.
- Pozwala tworzyć zarówno proste aplikacje okienkowe, jak i bardziej zaawansowane GUI z obsługą danych, animacji czy rysowania grafiki.

Przykład minimalnej aplikacji

- Tworzymy główne okno aplikacji (`JFrame`) z tytułem widocznym na pasku tytułowym.

```
JFrame frame = new JFrame("Hello Swing!");
```

- Określamy, że zamknięcie okna spowoduje zakończenie działania program (`System.exit(0)`).

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

- Ustawiamy rozmiar okna w pikselach: szerokość 300, wysokość 100.

```
frame.setSize(300, 100);
```

- Tworzymy etykietę tekstową i ustawiamy jej tekst.

```
JLabel label = new JLabel("Witaj w Swing!");
```

- Dodajemy etykietę do głównego obszaru zawartości (`content pane`) okna.

```
frame.getContentPane().add(label);
```

- Wyświetlamy okno. Bez tej linii GUI nie pojawi się na ekranie.

```
frame.setVisible(true);
```

Przykład minimalnej aplikacji – kompletny kod

```
import javax.swing.*;

public class HelloSwing {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Hello Swing!");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 100);

        JLabel label = new JLabel("Witaj w Swing!", SwingConstants.CENTER);
        frame.getContentPane().add(label);

        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

- `SwingConstants.CENTER` wyrównuje tekst do środka poziomo.
- `frame.setLocationRelativeTo(null);` wyśrodkowuje okno na ekranie użytkownika.
- Uwaga: Ten kod działa, ale nie używa Event Dispatch Thread (EDT).
- Zalecany sposób: użycie `SwingUtilities.invokeLater(...)`.

EDT (*Event Dispatch Thread*)

- EDT to specjalny wątek Swinga do obsługi zdarzeń i GUI
- Swing nie jest thread-safe – tylko EDT może modyfikować komponenty graficzne
- Wszystkie zdarzenia użytkownika (kliknięcia, wpisywanie) trafiają do EDT
- Kod GUI powinien być wykonywany w EDT – rysowanie, aktualizacja, zmiany
- Jeśli nie użyjemy EDT:
 - W aplikacji mogą wystąpić błędy graficzne: miganie, nieodświeżanie, wyjątki
 - Aplikacja może się zawiesić lub zachowywać losowo
- Zalecenie: zawsze należy używać `invokeLater()` lub `invokeAndWait()` do modyfikacji GUI

Minimalna aplikacja z EDT

```
import javax.swing.*;

public class HelloSwing {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new GuiStarter());
    }
}

class GuiStarter implements Runnable {
    public void run() {
        JFrame frame = new JFrame("Hello Swing!");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

- `Runnable` – reprezentuje zadanie do wykonania w osobnym wątku
 - Ma jedną metodę: `void run()`
- `SwingUtilities.invokeLater(...)`
 - Umieszcza kod w kolejce zdarzeń GUI
 - Gwarantuje, że kod GUI zostanie wykonany w bezpieczny sposób
 - Używamy go zawsze przy tworzeniu lub modyfikowaniu interfejsu Swing

Minimalna aplikacja z EDT z użyciem klasy anonimowej

```
import javax.swing.*;

public class HelloSwing {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                JFrame frame = new JFrame("Hello Swing!");
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                frame.setSize(300, 200);
                frame.setVisible(true);
            }
        });
    }
}
```

Klasa anonimowa

- To klasa bez nazwy, tworzona „w locie”
- Umożliwia szybkie utworzenie **jednorazowego obiektu** klasy implementującej interfejs (lub dziedziczącej po klasie)
- Piszemy implementację **w miejscu użycia**, bez tworzenia osobnej klasy

Minimalna aplikacja z EDT z użyciem wyrażenia lambda (Java 8+)

```
import javax.swing.*;

public class HelloLambda {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Hello Swing!");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(300, 200);
            frame.setVisible(true);
        });
    }
}
```

Lambda

- Skrócona składnia dla implementacji interfejsów funkcyjnych
- Pojawiła się w Java 8
- Zastępuje klasy anonimowe przy prostych interfejsach z jedną metodą (`Runnable` , `ActionListener` , ...)

Tworzenie interfejsu użytkownika

Przycisk i reakcja

Obsługa zdarzeń

- Tworzymy pojedynczy komponent przycisku (`JButton`)

```
JButton button = new JButton("Kliknij mnie");
```

- Dodajemy go do okna

```
frame.getContentPane().add(button);
```

- Dodajemy obsługę zdarzenia kliknięcia (`ActionListener`)

```
button.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        JOptionPane.showMessageDialog(frame, "Witaj świecie!");  
    }  
});
```

- Obsługę zdarzenia możemy dodać za pomocą funkcji lambda

```
button.addActionListener(e ->  
    JOptionPane.showMessageDialog(frame, "Kliknięto przycisk!")  
);
```

- `JOptionPane.showMessageDialog(...)` pokazuje proste okienko z tekstem

Okno z przyciskiem – kompletny kod

```
import javax.swing.*;

public class ButtonApp {
    public static void main(String[] args) {
        // Uruchomienie w EDT
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Przycisk w Swing");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(300, 100);

            JButton button = new JButton("Kliknij mnie");
            button.addActionListener(_ -> {
                JOptionPane.showMessageDialog(frame, "Witaj świecie!");
            });

            frame.getContentPane().add(button);
            frame.setLocationRelativeTo(null);
            frame.setVisible(true);
        });
    }
}
```

Mechanizm zdarzeń w Swing

- Zdarzenia to obiekty reprezentujące akcje użytkownika (np. kliknięcie przycisku, wpisanie tekstu).
- Elementy systemu zdarzeń:
 - `Source` – np. `JButton` (źródło zdarzenia, komponent GUI)
 - `Listener` – np. `ActionListener` (interfejs, reakcja na zdarzenie)
 - `Event` – np. `ActionEvent` (obiekt przekazany jako parametr do metody obsługi zdarzenia)

Jak reagować na zdarzenia?

1. Tworzymy komponent (np. `JButton`)
2. Dodajemy listenera: `addActionListener(...)`
3. Implementujemy metodę `actionPerformed()`

Zalecenia:

- Dla prostych akcji warto używać klas anonimowych lub funkcji lambda (od Java 8)
- Dla większych aplikacji – lepiej stosować osobne klasy lub kontrolery (wzorzec MVC)

Layout

Layouty

- Jeśli użyliśmy `JFrame.getContentPane().add(component)` bez określenia **layoutu**, Swing użył `BorderLayout`, a komponent dodany bez określenia pozycji trafił do centrum (`BorderLayout.CENTER`) – co powoduje, że rozciąga się na całe okno.
- Możemy wskazać, aby został umieszczony w innym miejscu `BorderLayout`

```
frame.getContentPane().add(button, BorderLayout.PAGE_START);
```

- Możemy też użyć innego layoutu, np. `FlowLayout` (Układa komponenty w jednej linii)

```
frame.setLayout(new FlowLayout());  
frame.add(button);
```

- Albo użyć bardziej zaawansowanego layoutu, np. panel z `BoxLayout` w osi Y (pionowo)

```
JPanel panel = new JPanel();  
panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));  
  
panel.add(Box.createVerticalGlue()); // Dodanie przestrzeni nad przyciskiem  
button.setAlignmentX(Component.CENTER_ALIGNMENT); // Dodanie przycisku, wyrównanie poziomo  
panel.add(button);  
panel.add(Box.createVerticalGlue()); // Dodanie przestrzeni pod przyciskiem  
  
frame.add(panel);
```

Formularz danych użytkownika

Tworzenie interfejsu robi się bardziej złożone, gdy w grę wchodzi więcej kontroltek

```
JFrame frame = new JFrame("Formularz - FlowLayout");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 150);

JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10)); // import java.awt.*;

panel.add(new JLabel("Imię:")); // etykieta
panel.add(new JTextField(10)); // pole tekstowe

panel.add(new JLabel("Nazwisko:"));
panel.add(new JTextField(10));

panel.add(new JLabel("Rola:"));
panel.add(new JComboBox<>(new String[] { "Użytkownik", "Admin" })); // rozwijana lista

panel.add(new JCheckBox("Aktywny")); // pole wyboru

frame.add(panel);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```

- Wersja z `FlowLayout` jest prosta ale niezbyt atrakcyjna.

Formularz danych użytkownika

Wersja z GridLayout

```
JFrame frame = new JFrame("Formularz - GridLayout");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 180);

JPanel panel = new JPanel(new GridLayout(4, 2, 10, 10));
panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

panel.add(new JLabel("Imię:"));
panel.add(new JTextField());

panel.add(new JLabel("Nazwisko:"));
panel.add(new JTextField());

panel.add(new JLabel("Rola:"));
panel.add(new JComboBox<>(new String[] { "Użytkownik", "Admin" }));

panel.add(new JLabel("")); // puste pole
panel.add(new JCheckBox("Aktywny"));

frame.add(panel);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```

- Lepsze wyrównanie (wszystko w siatce), ale wszystkie komórki mają równą szerokość.

Formularz danych użytkownika

Wersja z GridBagLayout

```
JFrame frame = new JFrame("Formularz - GridBagLayout");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(450, 200);

JPanel panel = new JPanel(new GridBagLayout());
panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(5, 5, 5, 5);
gbc.anchor = GridBagConstraints.WEST;

gbc.gridx = 0; gbc.gridy = 0; panel.add(new JLabel("Imię:"), gbc);
gbc.gridx = 1; panel.add(new JTextField(15), gbc);

gbc.gridx = 0; gbc.gridy = 1; panel.add(new JLabel("Nazwisko:"), gbc);
gbc.gridx = 1; panel.add(new JTextField(15), gbc);

gbc.gridx = 0; gbc.gridy = 2; panel.add(new JLabel("Rola:"), gbc);
gbc.gridx = 1; panel.add(new JComboBox<>(new String[] { "Użytkownik", "Admin" })), gbc);

gbc.gridx = 1; gbc.gridy = 3; panel.add(new JCheckBox("Aktywny"), gbc);

frame.add(panel); frame.setLocationRelativeTo(null); frame.setVisible(true);
```

- Najbardziej elastyczne i estetyczne rozwiązanie, idealne do realnych aplikacji.

Formularz danych użytkownika

GridBagLayout pozwala też na dynamiczne rozciąganie pól

```
// Wspólne ustawienia
gbc.insets = new Insets(5, 5, 5, 5);
gbc.anchor = GridBagConstraints.WEST;
gbc.fill = GridBagConstraints.HORIZONTAL;

// Etykieta: Imię
gbc.gridx = 0;
gbc.gridy = 0;
gbc.weightx = 0; // nie rozciągaj etykiety
panel.add(new JLabel("Imię:"), gbc);

// Pole tekstowe: Imię
gbc.gridx = 1;
gbc.weightx = 1; // pozwól na rozciąganie
panel.add(new JTextField(15), gbc);

// ...
```

- Pełna kontrola nad zachowaniem każdego komponentu.
- Można też wskazać by komponent zajmował więcej niż jedną kolumnę, np. `gbc.gridwidth = 2;`

Look and Feel

Look and Feel (L&F) – mechanizm, który decyduje o wyglądzie i zachowaniu graficznych komponentów interfejsu użytkownika — takich jak przyciski, pola tekstowe, menu itp.

- Metoda `UIManager.setLookAndFeel()` pozwala nam przełączyć L&F (należy to zrobić przed stworzeniem okna, najlepiej na początku `main()`)

```
try {  
    // wybieramy systemowy L&F  
    UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

- `UIManager` pozwala też ustawić domyślne właściwości poszczególnych komponentów, np. aby zmienić rozmiar tekstu:

```
UIManager.put("Label.font", new Font("SansSerif", Font.PLAIN, 16));  
UIManager.put("Button.font", new Font("SansSerif", Font.PLAIN, 16));  
UIManager.put("TextField.font", new Font("SansSerif", Font.PLAIN, 16));  
UIManager.put("List.font", new Font("SansSerif", Font.PLAIN, 16));  
UIManager.put("OptionPane.messageFont", new Font("SansSerif", Font.PLAIN, 16));  
UIManager.put("OptionPane.buttonFont", new Font("SansSerif", Font.PLAIN, 16));
```

Komponent listy

Lista osób

Wyświetlanie listy jest proste

```
JFrame frame = new JFrame("Lista osób");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(300, 200);

String[] osoby = {
    "Jan Kowalski",
    "Anna Nowak",
    "Piotr Zieliński",
    "Maria Wiśniewska"
};

JList<String> listaOsob = new JList<>(osoby);
JScrollPane scrollPane = new JScrollPane(listaOsob);

frame.getContentPane().add(scrollPane, BorderLayout.CENTER);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```

- Komponent `JList<>` pozwala na wyświetlanie danych w postaci listy
- `JScrollPane` pozwala na przewijanie, jeśli lista elementów jest zbyt długa by zmieścić się w oknie

Lista osób

`DefaultListModel` jako źródło danych

- `ListModel<>` umożliwia dynamiczne zarządzanie danymi listy – możemy dodawać, usuwać, modyfikować elementy w czasie działania programu.
- Używany zamiast przekazywania tablicy do `JList`, gdy potrzebujemy interakcji z danymi (np. w aplikacjach typu CRUD).
- Zaczynamy od utworzenia modelu danych (najprościej `DefaultListModel<>`)

```
DefaultListModel<String> model = new DefaultListModel<>();  
model.addElement("Jan Kowalski");  
model.addElement("Anna Nowak");  
model.addElement("Piotr Zieliński");
```

- Następnie tworzymy `JList<>` na podstawie modelu

```
JList<String> lista = new JList<>(model);
```

Lista osób

- Komponent `JList<>` pozwala również na przechowywanie obiektów dowolnej klasy, np. klasy `Person`, zamiast zwykłych `Stringów`.

```
class Person {  
    String firstName;  
    String lastName;  
  
    Person(String firstName, String lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
  
    @Override  
    public String toString() {  
        return firstName + " " + lastName;  
    }  
}
```

- `JList` działa z dowolnym typem obiektów, jeśli mają sensowną implementację `toString()`.
- Dzięki `toString()` wyświetlane będą dane użytkownika w czytelnej formie (np. „Anna Nowak”).

Lista osób

Z klasą `Person`

```
JFrame frame = new JFrame("Lista osób");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(300, 200);

// Model danych dla Person
DefaultListModel<Person> model = new DefaultListModel<>();
model.addElement(new Person("Jan", "Kowalski"));
model.addElement(new Person("Anna", "Nowak"));
model.addElement(new Person("Piotr", "Zieliński"));

// Lista z obiektami
JList<Person> lista = new JList<>(model);
JScrollPane scrollPane = new JScrollPane(lista);

frame.add(scrollPane, BorderLayout.CENTER);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```


Okna dialogowe

Lista z dodawaniem/edytowaniem/usuwaniem

Zaczniemy od interfejsu

- Nieco inna struktura pozwoli nam przenieść model i komponent do pól klasy – da nam to do nich dostęp w innych metodach

```
import javax.swing.*;
import java.awt.*;

public class PersonListEditor {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new PersonListEditor().createAndShowGUI());
    }

    private DefaultListModel<Person> model;
    private JList<Person> personList;

    private void createAndShowGUI() {
        // ...tu utworzymy GUI
    }
}
```

Lista z dodawaniem/edytowaniem/usuwaniem

Tworzymy model i komponenty kontrolek

```
// createAndShowGUI()
JFrame frame = new JFrame("Lista osób");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 300);

model = new DefaultListModel<>();
model.addElement(new Person("Jan", "Kowalski"));
model.addElement(new Person("Anna", "Nowak"));

personList = new JList<>(model);
JScrollPane scrollPane = new JScrollPane(personList);

JButton addButton = new JButton("Dodaj"); // przyciski pozwolą na dodawanie,
JButton editButton = new JButton("Edytuj"); // edycję
JButton deleteButton = new JButton("Usuń"); // i usuwanie osób

JPanel buttonPanel = new JPanel(); // osobny panel na przyciski
buttonPanel.add(addButton);
buttonPanel.add(editButton);
buttonPanel.add(deleteButton);

frame.add(scrollPane, BorderLayout.CENTER); // lista centralnie
frame.add(buttonPanel, BorderLayout.SOUTH); // a przyciski na dole
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```

Lista i dodawanie

Obsługa zdarzeń

- Tworząc komponenty, przypinamy obsługę zdarzeń:

```
// createAndShowGUI()  
// ...  
addButton.addActionListener(e -> addPerson());  
editButton.addActionListener(e -> editPerson());  
deleteButton.addActionListener(e -> deletePerson());  
// ...
```

- Działanie oddelegowujemy do metod klasy:

```
private void addPerson() {  
    Person p = showPersonDialog(null);  
    if (p != null) {  
        model.addElement(p);  
    }  
}
```

- Aby wczytać dane użytkownika, użyjemy okna dialogowego (w osobnej metodzie)

```
private Person showPersonDialog(Person existing) {  
    return ...;  
}
```

Lista z edycją

Obsługa zdarzeń

- Podobnie postępujemy z edycją, ale musimy upewnić się, że użytkownik wybrał osobę z listy.

```
private void editPerson() {  
    int index = personList.getSelectedIndex();  
    if (index >= 0) {  
        Person current = model.getElementAt(index);  
        Person updated = showPersonDialog(current);  
        if (updated != null) {  
            model.set(index, updated);  
        }  
    } else {  
        JOptionPane.showMessageDialog(null, "Wybierz osobę do edycji.");  
    }  
}
```

- To samo okno posłuży do edycji danych istniejącej osoby

Lista z usuwaniem

Obsługa zdarzeń

- Usuwanie wymaga tylko sprawdzenia, czy mamy co usunąć.

```
private void deletePerson() {  
    int index = personList.getSelectedIndex();  
    if (index >= 0) {  
        Person selected = model.get(index);  
        int confirm = JOptionPane.showConfirmDialog(null,  
            "Czy na pewno chcesz usunąć: " + selected + "?",  
            "Potwierdzenie",  
            JOptionPane.YES_NO_OPTION);  
        if (confirm == JOptionPane.YES_OPTION) {  
            model.remove(index);  
        }  
    } else {  
        JOptionPane.showMessageDialog(null, "Wybierz osobę do usunięcia.");  
    }  
}
```

- Przed usunięciem warto zapytać, czy użytkownik jest tego pewien.

Lista z dodawaniem i edycją

Otwieranie okna dialogowego

- `JOptionPane` pozwala na wyświetlenie prostego dialogu z własnymi kontrolkami

```
private Person showPersonDialog(Person existing) {
    JTextField firstNameField = new JTextField(15);
    JTextField lastNameField = new JTextField(15);

    if (existing != null) {
        // jeśli to edycja, wpisujemy dane który edytujemy
        firstNameField.setText(existing.firstName);
        lastNameField.setText(existing.lastName);
    }

    JPanel panel = new JPanel(new GridLayout(2, 2));
    panel.add(new JLabel("Imię:"));
    panel.add(firstNameField);
    panel.add(new JLabel("Nazwisko:"));
    panel.add(lastNameField);

    int result = JOptionPane.showConfirmDialog(
        null,                                     // komponent nadrzędny
        panel,                                    // treść okna dialogowego
        existing == null ? "Dodaj osobę" : "Edytuj osobę", // nagłówek
        JOptionPane.OK_CANCEL_OPTION);           // typ przycisków

    if (result == JOptionPane.OK_OPTION) { /* ... walidacja danych ... */ }
}
```

Lista z dodawaniem i edycją

c.d.

- Po zamknięciu okna dialogowego, walidujemy dane i zwracamy wynik

```
private Person showPersonDialog(Person existing) {  
    ...  
  
    int result = JOptionPane.showConfirmDialog(null, panel,  
        existing == null ? "Dodaj osobę" : "Edytuj osobę",  
        JOptionPane.OK_CANCEL_OPTION);  
  
    if (result == JOptionPane.OK_OPTION) {  
  
        String imie = firstNameField.getText().trim();  
        String nazwisko = lastNameField.getText().trim();  
  
        if (!imie.isEmpty() && !nazwisko.isEmpty()) {  
            return new Person(imie, nazwisko);  
        } else {  
            JOptionPane.showMessageDialog(null, "Wszystkie pola muszą być wypełnione.");  
        }  
    }  
    return null; // jeśli użytkownik zrezygnował  
}
```

- Można również użyć klasy `JDialog` (lepsza kontrola, możliwość użycia własnego układu)

JDialog

- Własny dialog to klasa dziedzicząca z JDialog

```
public class CustomPersonDialog extends JDialog {
    // tworzymy komponenty interfejsu
    private JTextField firstNameField = new JTextField(15);
    private JTextField lastNameField = new JTextField(15);
    private boolean confirmed = false;

    public CustomPersonDialog(JFrame parent) {
        super(parent, "Dodaj osobę", true); // okno modalne - blokujące
        // budujemy zawartość okna
    }

    // aby umożliwić sprawdzenie czy okno zamknięto przyciskiem OK
    public boolean isConfirmed() {
        return confirmed;
    }

    // aby umożliwić odczyt danych z dialogu
    public String getFirstName() {
        return firstNameField.getText().trim();
    }

    public String getLastName() {
        return lastNameField.getText().trim();
    }
}
```

JDialog

- Zawartość okna budujemy tak, jak w wypadku okna głównego

```
setLayout(new BorderLayout());
JPanel formPanel = new JPanel(new GridLayout(2, 2, 5, 5));

formPanel.add(new JLabel("Imię:"));
formPanel.add(firstNameField);
formPanel.add(new JLabel("Nazwisko:"));
formPanel.add(lastNameField);

JButton okButton = new JButton("OK");
JButton cancelButton = new JButton("Anuluj");

JPanel buttons = new JPanel();
buttons.add(okButton);
buttons.add(cancelButton);

add(formPanel, BorderLayout.CENTER);
add(buttons, BorderLayout.SOUTH);
pack();
setLocationRelativeTo(parent);
```

JDialog

- Tak samo też dołączamy obsługę zdarzeń

```
okButton.addActionListener(e -> {  
    confirmed = true;  
    setVisible(false);  
});  
  
cancelButton.addActionListener(e -> {  
    setVisible(false);  
});
```

JDialog

- Użycie (w głównym oknie)

```
JButton openDialog = new JButton("Dodaj osobę");

openDialog.addActionListener(e -> {
    // Tworzymy obiekt
    CustomPersonDialog dialog = new CustomPersonDialog(frame);
    // Otwieramy okno
    dialog.setVisible(true);

    // Po zamknięciu możemy sprawdzić, czy użytkownik zatwierdził operację
    if (dialog.isConfirmed()) {
        // Odczytujemy wprowadzone dane
        String imie = dialog.getFirstName();
        String nazwisko = dialog.getLastName();
        JOptionPane.showMessageDialog(frame, "Dodano: " + imie + " " + nazwisko);
    }
});
```

Lista z własnym rendererem

- Można wskazać własnego renderera elementów listy – pozwoli to np. na stworzenie listy z bardziej złożonymi kaflami reprezentującymi osoby.

```
personList = new JList<>(model);  
personList.setCellRenderer(new PersonRenderer());
```

- Prosty renderer:

```
static class PersonRenderer extends JPanel implements ListCellRenderer<Person> {  
    private final JLabel nameLabel = new JLabel();  
    private final JLabel surnameLabel = new JLabel();  
  
    public PersonRenderer() {  
        // tworzymy złożony komponent  
        setLayout(new BorderLayout(2, 2));  
        nameLabel.setFont(nameLabel.getFont().deriveFont(Font.BOLD));  
        surnameLabel.setForeground(Color.DARK_GRAY);  
        surnameLabel.setFont(surnameLabel.getFont().deriveFont(Font.PLAIN, 12f));  
        add(nameLabel, BorderLayout.NORTH);  
        add(surnameLabel, BorderLayout.CENTER);  
        setBorder(BorderFactory.createEmptyBorder(4, 4, 4, 4));  
    }  
  
    ...  
}
```

Renderer

c.d.

```
...

// Tu powinniśmy zaktualizować komponent (zmienić zawartość, kolory, etc.)
@Override
public Component getListCellRendererComponent(JList<? extends Person> list,
                                              Person value,
                                              int index,
                                              boolean isSelected,
                                              boolean cellHasFocus) {

    nameLabel.setText(value.firstName);
    surnameLabel.setText(value.lastName);

    if (isSelected) {
        setBackground(list.getSelectionBackground());
        setForeground(list.getSelectionForeground());
    } else {
        setBackground(list.getBackground());
        setForeground(list.getForeground());
    }
    setOpaque(true);
    return this;
}
```

Alternatywne rozwiązanie – JTable

- JTable pozwala na wyświetlanie danych w kolumnach
- Kontakt kontrolki z danymi następuje przez klasę modelu:

```
private final List<Person> people = new ArrayList<>();
private final PersonTableModel tableModel = new PersonTableModel();

// Model tabeli
class PersonTableModel extends AbstractTableModel {

    @Override
    public int getRowCount() { }

    @Override
    public int getColumnCount() { }

    @Override
    public String getColumnName(int column) { }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) { }

    @Override
    public boolean isCellEditable(int row, int col) { }
}
```

Model tabeli

```
class PersonTableModel extends AbstractTableModel {
    private final String[] columnNames = {"Imię", "Nazwisko"};

    @Override
    public int getRowCount() { return people.size(); }

    @Override
    public int getColumnCount() { return columnNames.length; }

    @Override
    public String getColumnName(int column) { return columnNames[column]; }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Person p = people.get(rowIndex);
        return columnIndex == 0 ? p.firstName : p.lastName;
    }

    // edycja tylko przez dialog
    @Override
    public boolean isCellEditable(int row, int col) { return false; }
}
```


JTable z osobami

- Tworzenie okna i jego komponentów wygląda podobnie:

```
JTable table = new JTable(tableModel);
JScrollPane scrollPane = new JScrollPane(table);

frame.add(scrollPane, BorderLayout.CENTER);
```

- Po edycji danych, należy powiadomić komponent tabeli o zmianie modelu

```
private void addPerson() {
    Person p = showPersonDialog(null);
    if (p != null) {
        people.add(p);
        tableModel.fireTableDataChanged();
    }
}
```

Edycja w tabeli

- Aby umożliwić edycję należy w modelu zwrócić `true` z `isCellEditable()`

```
public boolean isCellEditable(int row, int col) {  
    return true; // lub np. col != 0  
}
```

- Następnie nadpisujemy `setValueAt()`

```
public void setValueAt(Object value, int row, int col) {  
    Person p = people.get(row);  
    if (col == 0) {  
        p.firstName = value.toString();  
    } else if (col == 1) {  
        p.lastName = value.toString();  
    }  
    fireTableCellUpdated(row, col);  
}
```

DefaultTableModel

Do prostych przypadków można też użyć DefaultTableModel

```
// Tworzenie modelu z nazwami kolumn
DefaultTableModel model = new DefaultTableModel(new Object[]{"Imię", "Nazwisko"}, 0);
JTable table = new JTable(model);

// Dodanie osoby
model.addRow(new Object[]{"Anna", "Nowak"});

// Usunięcie osoby
int selected = table.getSelectedRow();
if (selected >= 0) model.removeRow(selected);
```

- Dane można dodawać również bezpośrednio w tabeli (bez okien dialogowych):

```
// Przycisk "Dodaj" – dodaje pusty wiersz
JButton addButton = new JButton("Dodaj");
addButton.addActionListener(e -> model.addRow(new Object[]{"", ""}));
```

Zapis i odczyt danych

Jak teraz zapisać te dane do pliku?

- Najprostsze rozwiązanie to serializacja
- Serializacja to proces zamiany obiektu Java w ciąg bajtów (np. do zapisu na dysku lub przestania przez sieć), a deserializacja to odtworzenie obiektu z tych bajtów.
- W Javie realizuje się to przez implementację interfejsu `Serializable` ...

```
class Person implements Serializable { ... }
```

- ...i użycie klas takich jak `ObjectOutputStream` i `ObjectInputStream`.

```
List<Person> people = ...

try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {
    out.writeObject(people);
    JOptionPane.showMessageDialog(null, "Zapisano dane do pliku.");
} catch (IOException e) {
    JOptionPane.showMessageDialog(null, "Błąd zapisu: " + e.getMessage());
}
```

- Do pliku wrzucamy całą strukturę danych (np. listę obiektów)

Zapis i odczyt danych

- Odczyt jest równie prosty co zapis

```
File file = new File(FILE_NAME);
if (!file.exists()) return;

try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(file))) {
    people = (List<Person>) in.readObject();
} catch (IOException | ClassNotFoundException e) {
    JOptionPane.showMessageDialog(null, "Błąd odczytu: " + e.getMessage());
}
```

- Serializacja jest prosta w użyciu, ale ma swoje ograniczenia
 - Wczytanie nieznanych lub spreparowanych bajtów przez `ObjectInputStream` może prowadzić do ataków deserializacyjnych (Remote Code Execution). Nie należy używać serializacji do komunikacji między aplikacjami lub do odczytu danych z niezaufanego źródła.
 - Zmiana w klasie (np. dodanie pola) może uniemożliwić deserializację starych danych.
 - Dane serializowane w Javie są nieczytelne dla innych języków.

Pytanie o nazwę pliku

- Najprostsze rozwiązanie to użycie `JOptionPane` do wyświetlenia dialogu tekstowego (`showInputDialog`):

```
String fileName = JOptionPane.showInputDialog(frame,
                                           "Podaj nazwę pliku:",
                                           "Zapisz do pliku",
                                           JOptionPane.PLAIN_MESSAGE);

if (fileName != null && !fileName.trim().isEmpty()) {
    File file = new File(fileName.trim());
    // użyj file do zapisu lub odczytu
}
```

- W aplikacji z GUI lepiej użyć dedykowanego dialogu otwarcia pliku

```
JFileChooser chooser = new JFileChooser();
int result = chooser.showSaveDialog(frame); // lub showOpenDialog
if (result == JFileChooser.APPROVE_OPTION) {
    File file = chooser.getSelectedFile();
    // teraz mamy ścieżkę do pliku: file.getAbsolutePath()
}
```

Rysowanie

Rysowanie i animacja

- Za rysowanie i animację będzie odpowiadał nasz panel - BallPanel

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class BouncingBallApp {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Odbijająca się piłka");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(500, 400);
            frame.setLocationRelativeTo(null);

            BallPanel panel = new BallPanel();
            frame.add(panel);
            frame.setVisible(true);

            panel.startAnimation();
        });
    }
}
```


BallPanel

```
class BallPanel extends JPanel implements ActionListener {
    private int x = 100, y = 100;    // pozycja piłki
    private int dx = 2, dy = 3;      // prędkość (zmiana pozycji)
    private final int RADIUS = 20;   // promień piłki
    private final Timer timer;

    public BallPanel() {
        setBackground(Color.WHITE);
        timer = new Timer(10, this); // Co 10 ms wywołuj actionPerformed (czyli 100 FPS)
    }

    public void startAnimation() {
        timer.start(); // uruchamia powtarzalne zdarzenia (tyknięcia zegara)
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g); // czyszczenie tła
        g.setColor(Color.RED);
        g.fillOval(x, y, RADIUS * 2, RADIUS * 2); // rysowanie piłki jako okrąg
    }

    @Override
    public void actionPerformed(ActionEvent e) { ... }
}
```

Animacja

```
class BallPanel extends JPanel implements ActionListener {  
    ...  
  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        int width = getWidth();  
        int height = getHeight();  
  
        // Zmiana pozycji  
        x += dx;  
        y += dy;  
  
        // Odbicie od krawędzi  
        if (x <= 0 || x + RADIUS * 2 >= width) {  
            dx = -dx;  
        }  
        if (y <= 0 || y + RADIUS * 2 >= height) {  
            dy = -dy;  
        }  
  
        repaint(); // wywołanie ponownego narysowania (trafi do paintComponent)  
    }  
}
```